12.16

Suppose that two registers contain the following hexadecimal values: AB0890C2,4598EE50. What is the result of adding them using MMX instructions:a. packed byte.b. packed word.Assume saturation arithmetic is not used.

=> a) packed byte: Add 1 byte at a time → 8 bit → 2 digit here

C → D → E → F → 0 → 1

carry no carry

AB 08 90 C2
45 98 EE 50

F0 A0 7E 12

b) packed word: Add 2 bytes (16 bit → 4 digits

no carry carries carry

AB 08 90 C2
45 98 EE 50

F0 A0 7F 12

12.17

Appendix I points out that there are nostack-oriented instructions in an instruction setif the stack is to be used only by the processor for such purposes as procedure handling.How can the processor use a stack for any purpose withoutstack-oriented instructions?

=> if a proc used a stack only for subroutine handling, it doesn't require explicit stack oriented instrs like PUSH & POP. Instead the stack is implicitly managed by call & return instrs.

The proc automatically manages the stack only when needed

Call instrs: → Saves the return address (current instr pointer) onto the stack.

→ Jumps to subroutine's starting address.

Return instr: → retrieves the saved return addrs from the stack.

→ Jumps back to the caller func.

Sure! Here's a simple assembly example to illustrate how a processor manages the stack implicitly for function calls:

```assembly
; Main program
START:
    ; Call function1
    CALL function1

    ; Do something after function call
    MOV AX, 0x4C00  ; Exit the program
    INT 0x21        ; DOS interrupt to exit

; Function 1
function1:
```

```
    ; Call function2
    CALL function2

    ; Return from function1
    RET

; Function 2
function2:
    ; Do something in function2
    MOV BX, 0x1234  ; Sample operation

    ; Return from function2
    RET
```

Breakdown:
1. **CALL function1**:
   - The `CALL` instruction saves the return address (the next instruction after `CALL function1`, which is the `MOV AX, 0x4C00` instruction) onto the stack, then jumps to `function1`.

2. **Inside function1**:
   - The `CALL function2` instruction behaves the same: it saves the return address (the next instruction, `RET`), then jumps to `function2`.

3. **Inside function2**:
   - The `MOV BX, 0x1234` instruction is executed, and then the `RET` instruction pops the return address from the stack and jumps back to `function1`.

4. **Return to Main**:
   - After `function1` finishes, its `RET` pops the return address (which points to `MOV AX, 0x4C00`) and control returns to the main program.

In this case, the stack is implicitly managed: the `CALL` instruction pushes the return address, and the `RET` instruction pops it, allowing the processor to return to the correct location without needing explicit `PUSH` or `POP` instructions for general-purpose use.

12.18

Convert the following formulas fromreverse Polish to infix:

a. AB+C+DX

=(A+B+C)XD

X
D
+
C
(A+B+C)XD

b) AB/CD/+

=(A/B)+(C/D)

c) ABCDE+XX/

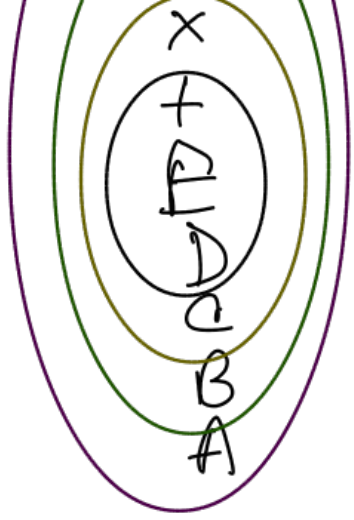
=>

A/BCX(D+E)



↓) ABCDE + F/ + G − H/ X +

A+
(B* (C + (D + E) / F − G) / H)



12.19 A + B + C + D + E

=> A B C D E + + + + ?

AB + C + D + E +

b) (A + B) X (C + D) + E

= AB + CD + X E +

c) (A X B) + (C X D) + E

AB X CD X + E +

d) (A − B) X (((C − D X E) / F) / G X H

= AB − CDEX − XF / G / H X X

H

Multiply with AB- then X H ✓

12.20 A + B − C => AB + C − => (A + B) − C

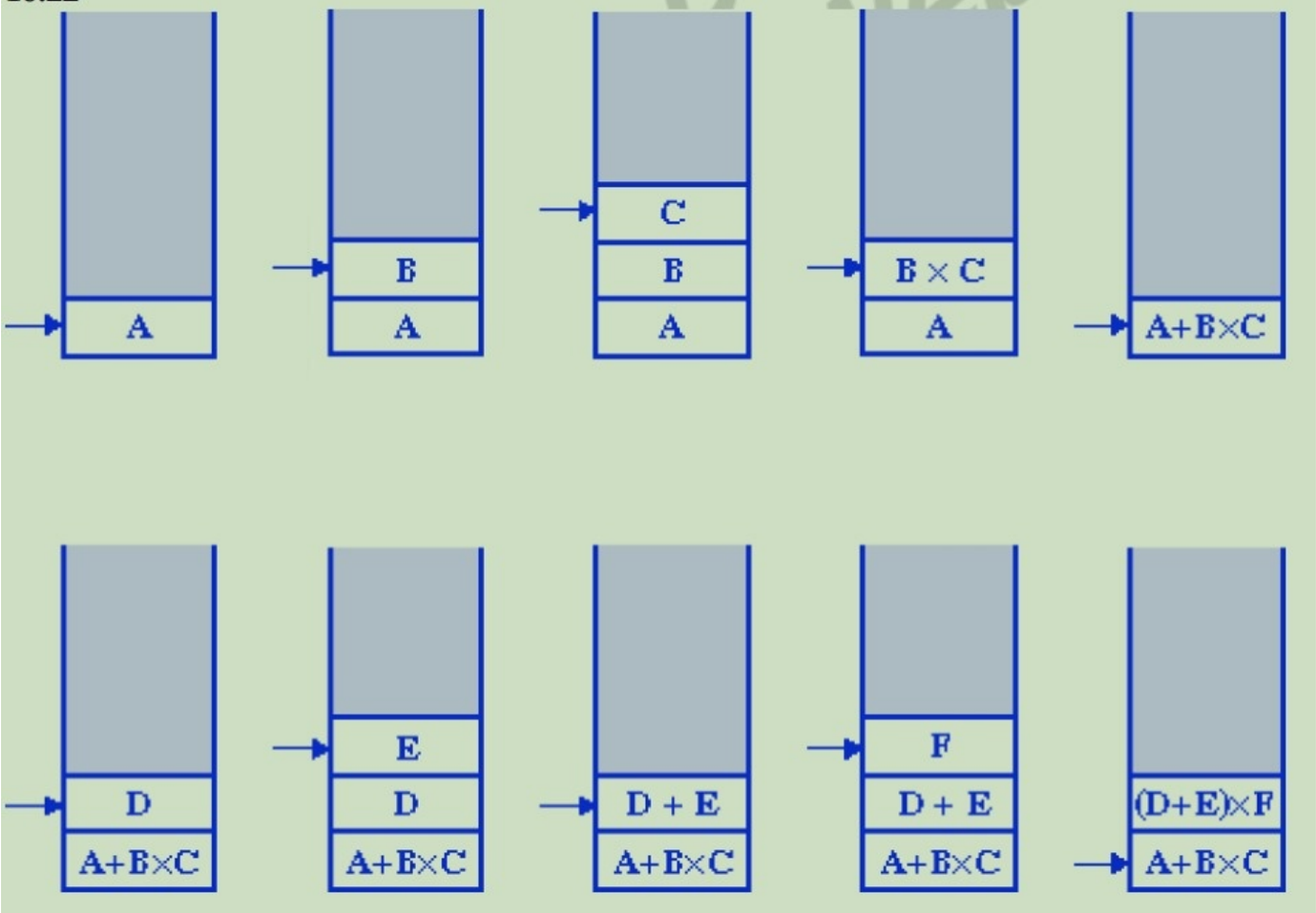
order matters for rounding & truncation

12.21

Input	Output	Stack (top on right)
(A – B) / (C + D × E)	empty	empty
A – B) / (C + D × E)	empty	(
– B) / (C + D × E)	A	(
B) / (C + D × E)	A	(–
) / (C + D × E)	A B	(–
/ (C + D × E)	A B –	empty
(C + D × E)	A B –	/
C + D × E)	A B –	/ (
+ D × E)	A B – C	/ (
D × E)	A B – C	/ (+
× E)	A B – C D	/ (+
E)	A B – C D	/ (+ ×
)	A B – C D E	/ (+ ×
empty	A B – C D E × +	/
empty	A B – C D E × + /	empty

12.22

10.22



The final step combines the top two stack elements using the + operator.

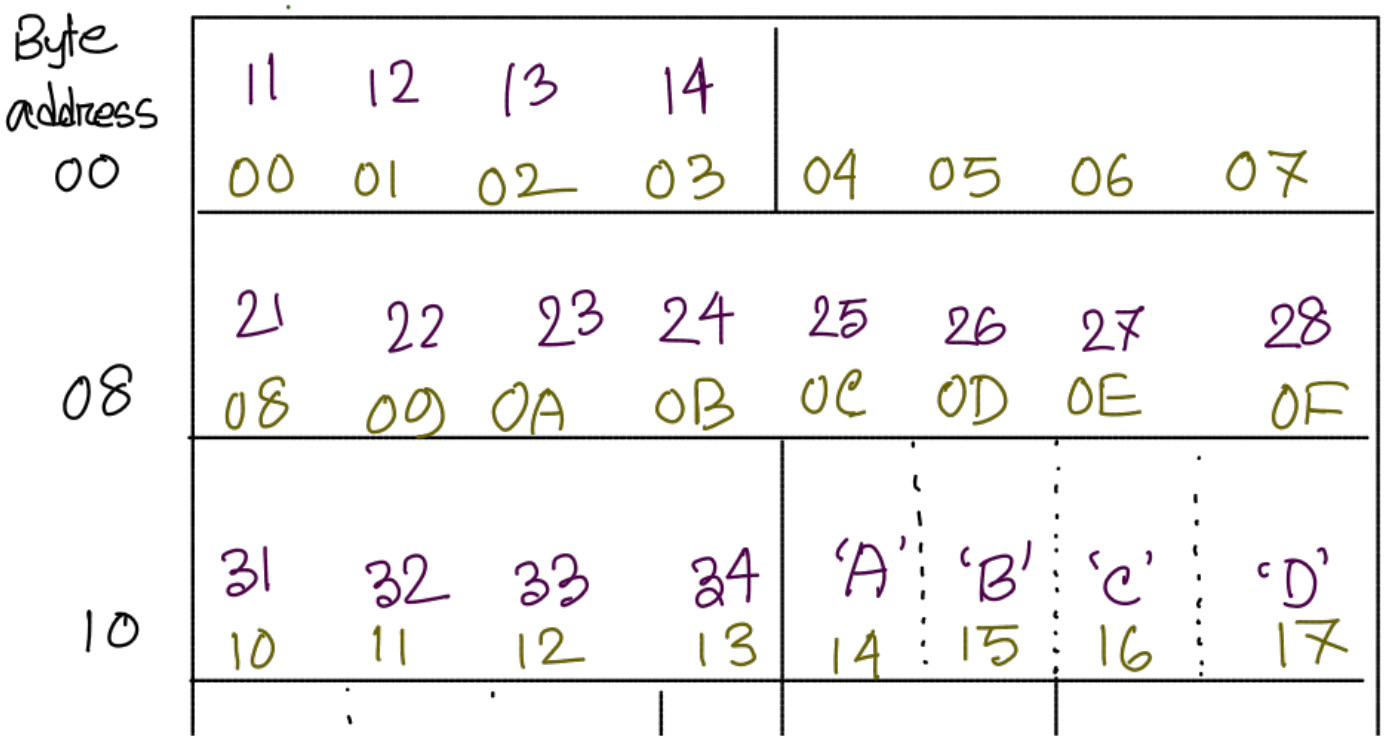
12.23

Redraw thelittle-endian layout in Figure 12.13 so that the bytes appear as numberedin thebig-endian layout. That is, show memory in 64-bit rows, with the bytes listed leftto right, top to bottom.

=> original :

```
struct
{
    int     a;      // 0x1112_1314                      word
    int     pad;    //
    double  b;      // 0x2122_2324_2526_2728            doubleword
    char*   c;      // 0x3132_3334                      word
    char    d[7];   // 'A', 'B', 'C', 'D', 'E', 'F', 'G'   byte array
    short   e;      // 0x5152                           halfword
    int     f;      // 0x6162_6364                      word
} s;
```

C++



	'E'	'F'	'G'		51	52		
18	18	19	1A	1B	1C	1D	1E	1F
	61	62	63	64				
20	20	21	22	23				

Byte address								
	14							
00	00	01	02	03	04	05	06	07
	28							
08	08	09	0A	0B	0C	0D	0E	0F
	34				'A'			
10	10	11	12	13	14	15	16	17

	'E'							
18	18	19	1A	1B	1C	1D	1E	1F
	'A'							
20	20	21	22	23				

12.24

Please Do not post on Web

Coa willStall Soln

BE	11	12	13	14	15	16	17	18
	00	01	02	03	04	05	06	07
LE	11	12	13	14	15	16	17	18
	07	06	05	04	03	02	01	00
BE	11	12	13	14	15	16	17	18
	00	01	02	03	04	05	06	07
LE	15	16	17	18	11	12	13	14
	07	06	05	04	03	02	01	00
BE	11	12	13	14	15	16	17	18
	00	01	02	03	04	05	06	07
LE	17	18	15	16	13	14	11	12
	07	06	05	04	03	02	01	00

This document is available on studocu
Downloaded by Shara Ud (shara1332875@gmail.com)

Computer Organiz...

e little-endian layout in Figure 12.13 so that the bytes app
endian layout. That is, show memory in 64-bit rows, with th
p to bottom.
lowing data structures, draw the big-endian and little-endi
of Figure 12.13, and comment on the results.

```
a.  struct {
        double i;       //0x1112131415161718
    } s1;
b.  struct {
        int i;          //0x11121314
        int j;          //0x15161718
    } s2;
c.  struct {
        short i;        //0x1112
        short j;        //0x1314
        short k;        //0x1516
        short l;        //0x1718
    } s3;
```

ower architecture specification does not dictate how a proces
-endian mode. It specifies only the view of memory a proc
ating in little-endian mode. When converting a data structur
lian, processors are free to implement a true byte-swapping
ort of an address modification mechanism. Current Power
-endian machines and use address modification to treat data
sider the structure s defined in Figure 12.13. The layout i
the figure shows the structure s as seen by the processor
mpiled in little-endian mode, its layout in memory is show

Ch 12 math

Byte address

14							
00	01	02	03	04	05	06	07
08							